

1. Login to Redshift

Connection between EC2 and Redshift is established

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Thu Aug 20 20:03:52 2026 from 195.56.119.209
[ec2-user@ip-172-31-9-34 ~]$ psql -h data-bi-lab-redshift-cluster-3.cettexdsxw3v.eu-central-1.redshift.amazonaws.com -v
schema=public -p 5439 -U dilab_student83 -d dev_dilab
Password for user dilab_student83:
psql (15.18, server 8.0.2)
WARNING: psql major version 15, server major version 8.0.
Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

dev_dilab=> |

dev_dilab=> SELECT * FROM information_schema.tables LIMIT 5;
 table_catalog | table_schema | table_name | table_type | self_referencing_column_name | refo
-----+-----+-----+-----+-----+-----+
 dev_dilab    | information_schema | information_schema_catalog_name | VIEW | |
 dev_dilab    | information_schema | applicable_roles | VIEW | |
 dev_dilab    | information_schema | check_constraints | VIEW | |
 dev_dilab    | information_schema | column_domain_usage | VIEW | |
 dev_dilab    | information_schema | column_privileges | VIEW | |
(5 rows)

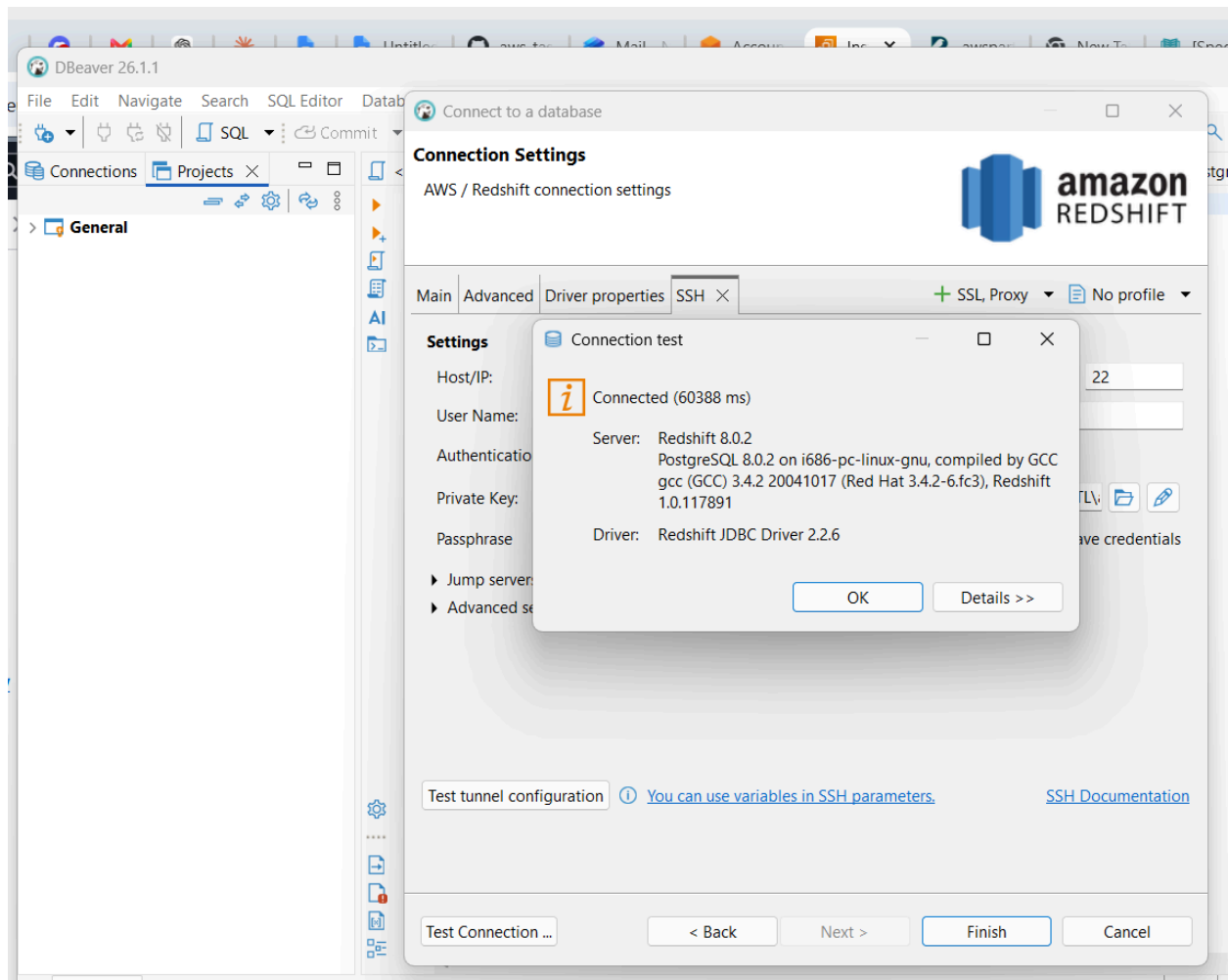
dev_dilab=>
```

Postgres status:

```
[ec2-user@ip-172-31-9-34 ~]$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL database server
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-22 10:49:29 UTC; 46min ago
     Process: 2445 ExecStartPre=/usr/libexec/postgresql-check-db-dir postgresql (code=exited, status=0/SUCCESS)
    Main PID: 2452 (postgres)
       Tasks: 7 (limit: 1113)
      Memory: 32.2M
         CPU: 386ms
    CGroup: /system.slice/postgresql.service
            └─2452 /usr/bin/postgres -D /var/lib/pgsql/data
              └─2476 "postgres: logger "
                └─2477 "postgres: checkpointer "
                  └─2478 "postgres: background writer "
                    └─2481 "postgres: walwriter "
                      └─2482 "postgres: autovacuum launcher "
                        └─2483 "postgres: logical replication launcher "
```

```
Aug 22 10:49:29 ip-172-31-9-34.eu-central-1.compute.internal systemd[1]: Starting postgresql.service - PostgreSQL datab
Aug 22 10:49:29 ip-172-31-9-34.eu-central-1.compute.internal postgres[2452]: 2026-08-22 10:49:29.315 UTC [2452] LOG: r
Aug 22 10:49:29 ip-172-31-9-34.eu-central-1.compute.internal postgres[2452]: 2026-08-22 10:49:29.315 UTC [2452] HINT: >
Aug 22 10:49:29 ip-172-31-9-34.eu-central-1.compute.internal systemd[1]: Started postgresql.service - PostgreSQL databa
lines 1-21/21 (END)
```

Connection settings are set



2. COPY

Tables are created on Readshift

The screenshot shows the Readshift SQL editor with a database connection named 'dev_dilab' and a schema 'dilab_student83@dev_dilab'. The SQL editor contains a CREATE TABLE statement for 'fct_orders_dd' and a query that counts records in three tables: 'dim_channels', 'dim_products', and 'fct_orders_dd'. The query results are displayed in a grid below the editor.

```
CREATE TABLE fct_orders_dd (
  source_system VARCHAR(50) NOT NULL,
  source_entity VARCHAR(50) NOT NULL,
  order_src_id VARCHAR(100) NOT NULL,
  insert_dt DATE NOT NULL,
  update_dt DATE NOT NULL,
  CONSTRAINT pk_fct_orders_dd PRIMARY KEY (order_src_id, source_system, event_dt)
);

SELECT 'dim_channels' AS table_name, COUNT(*) AS total_rows FROM dilab_student83.dim_channels
UNION ALL
SELECT 'dim_products', COUNT(*) FROM dilab_student83.dim_products
UNION ALL
SELECT 'fct_orders_dd', COUNT(*) FROM dilab_student83.fct_orders_dd;
```

table_name	total_rows
dim_channels	0
dim_products	0
fct_orders_dd	0

Data copied to Readshift

The screenshot shows the Readshift Output window with three messages indicating successful data loads into the tables 'dim_channels', 'dim_products', and 'fct_orders_dd'. Below the output window, a statistics table is displayed.

Load into table 'dim_channels' completed, 6 record(s) loaded successfully.
Load into table 'dim_products' completed, 5000 record(s) loaded successfully.
Load into table 'fct_orders_dd' completed, 108815 record(s) loaded successfully.

Name	Value
Execute time	37s
Start time	Sat Aug 22 16:04:57 EEST 2026
Finish time	Sat Aug 22 16:05:34 EEST 2026
Query	COPY dilab_student83.fct_orders_dd FROM 's3://nadiia-diachenko-dwh-bucket/di_dwh_database/schema_bl_dm/fct_orders_dd/fct_orders_dd.csv'

<pre> SELECT "schema" AS schema_name, "table" AS table_name, tbl_rows AS total_rows, size AS size_in_mb FROM svv_table_info WHERE "schema" = 'dilab_student83' AND "table" IN ('dim_channels', 'dim_products', 'fct_orders_dd'); </pre>				
Statistics 1 Statistics 2 svv_table_info 3				
LECT "schema" AS schema_name, " Enter a SQL expression to filter results (use Ctrl+Space)				
	AZ schema_name	AZ table_name	123 total_rows	123 size_in_mb
	dilab_student83	dim_channels	6	24
	dilab_student83	fct_orders_dd	108,815	120
	dilab_student83	dim_products	5,000	26

Table-Level Distribution Style, Sort Keys, and Size

Distribution Style

Dimension Tables (dim_channels, dim_products) automatically assigned AUTO(ALL) because these tables contain a small number of rows (6 and 5,000 rows respectively), Redshift replicates a full copy of each table onto every compute node to eliminate network data shuffling during SQL joins.

Fact Table (fct_orders_dd). AUTO(KEY(order_src_id)) was assigned by Redshift. With DISTSTYLE AUTO, Redshift automatically selected a distribution key based on the table's characteristics and workload considerations.

Sort Keys

Dimension Tables: Assigned AUTO(SORTKEY) with no active column sequence.

Fact Table: Redshift automatically picked event_dt as the primary sort key (sortkey = 1 in pg_table_def). This physically organizes rows on disk by date, optimizing time-series range filtering.

Statistics 1 Statistics 2 svv_table_info 3					
SELECT "table" AS table_name, diststyle Enter a SQL expression to filter results (use Ctrl+Space)					
	AZ table_name	AZ diststyle	AZ sortkey1	123 size_in_mb	123 total_rows
1	dim_channels	AUTO(ALL)	AUTO(SORTKEY)	24	6
2	dim_products	AUTO(ALL)	AUTO(SORTKEY)	26	5,000
3	fct_orders_dd	AUTO(KEY(order_src_id))	AUTO(SORTKEY)	120	108,815

3. Column Compression

!Notes

Initially, Redshift applied automatically sort and distribution keys across the tables (because I created plain tables) that appeared to be different to each table and show strange results:

"table" AS table_name, tbl_rows AS | Enter a SQL expression to filter results (use Ctrl+Space)

A-Z table_name	123 total_rows	123 size_in_mb	A-Z is_encoded
fct_orders_dd_analyzedcomp	108,815	120	Y
fct_orders_dd	108,815	120	Y, AUTO(ENCODE)
fct_orders_dd_withoutcomp	108,815	36	N

After recreating and standardizing the keys across 3 tables, I received results that I did not expect to see also - they were the same!

ble_info 3	Results 4	pg_table_def 3	svv_table_info 4	stv_blocklist(+) 5	svv_table_info 6	pg_table_def 7	fct
T "table" AS table_name, tbl_rows AS total_							
A-Z table_name	123 total_rows	123 size_in_mb	A-Z has_compression_encoding	A-Z diststyle	A-Z sortkey1		
fct_orders_dd	108,815	240	Y, AUTO(ENCODE)	KEY(order_surr_id)	event_dt		
fct_orders_dd_analyzedcomp	108,815	240	Y	KEY(order_surr_id)	event_dt		
fct_orders_dd_withoutcomp	108,815	240	N	KEY(order_surr_id)	event_dt		

So, what I concluded from the investigation of the issue is that, probably, this is because Redshift allocates storage in fixed-size blocks, with a minimum of one block per column per slice and at my tables size, both raw and compressed data already fit within that minimum, so compression savings weren't visible at the block level.

The reporting provided on standardized the keys across 3 tables:

Default state:

During the initial COPY execution Redshift applied suggested encodings:

-az64 Encoding applied to all high-cardinality numeric and temporal attributes: including BIGINT keys, DATE attributes (event_dt, insert_dt), and decimal metrics (NUMERIC(10,2)).

-lzo Encoding applied to all text and string descriptors (VARCHAR(50), VARCHAR(100), VARCHAR(255)).

-no(none) encoding applied to event_dt

Editor Database Window Help

Commit Rollback Auto dev_dilab dilab_student83@dev_dilab

postgres * <postgres> dim_customers_scd <postgres> Script <dev_dilab> Script-18

svv_table_info 3 Results 4 pg_table_def 3

SELECT tablename, "column" AS column_name, t

	A-Z tablename	A-Z column_name	data_type	A-Z compression_type
1	fct_orders_dd	order_surr_id	bigint	az64
2	fct_orders_dd	event_dt	date	none
3	fct_orders_dd	product_surr_id	bigint	az64
4	fct_orders_dd	customer_surr_id	bigint	az64
5	fct_orders_dd	sales_rep_surr_id	bigint	az64
6	fct_orders_dd	city_surr_id	bigint	az64
7	fct_orders_dd	subchannel_surr_id	bigint	az64
8	fct_orders_dd	order_status	character varying(50)	lzo
9	fct_orders_dd	returned	character varying(10)	lzo
10	fct_orders_dd	payment_method	character varying(50)	lzo
11	fct_orders_dd	fct_unit_price	numeric(10,2)	az64
12	fct_orders_dd	fct_quantity	bigint	az64
13	fct_orders_dd	fct_discount_percent	bigint	az64
14	fct_orders_dd	fct_discount_amount	numeric(10,2)	az64
15	fct_orders_dd	fct_shipping_cost	numeric(10,2)	az64
16	fct_orders_dd	fct_tax_amount	numeric(10,2)	az64
17	fct_orders_dd	fct_order_amount	numeric(10,2)	az64
18	fct_orders_dd	fct_cost_amount	numeric(10,2)	az64
19	fct_orders_dd	fct_profit_margin_percent	numeric(10,2)	az64
20	fct_orders_dd	fct_profit_amount	numeric(10,2)	az64
21	fct_orders_dd	fct_net_amount	numeric(10,2)	az64
22	fct_orders_dd	transaction_id	character varying(100)	lzo
23	fct_orders_dd	source_system	character varying(50)	lzo
24	fct_orders_dd	source_entity	character varying(50)	lzo
25	fct_orders_dd	order_src_id	character varying(100)	lzo
26	fct_orders_dd	insert_dt	date	az64
27	fct_orders_dd	update_dt	date	az64

The fct_orders_dd_withoutcomp is created

Commit Rollback Auto dev_dilab dilab_student83@dev_dilab				
postgres * <postgres> di... <postgres> Scri... <dev_dilab> Sc... <postgres> dim...				
svv_table_info 3 Results 4 pg_table_def 3 pg_table_def 4				
SELECT tablename, "column" AS column_name, t				
	AZ tablename	AZ column_name	data_type	AZ compression_type
1	fct_orders_dd_withoutcomp	order_surr_id	bigint	none
2	fct_orders_dd_withoutcomp	event_dt	date	none
3	fct_orders_dd_withoutcomp	product_surr_id	bigint	none
4	fct_orders_dd_withoutcomp	customer_surr_id	bigint	none
5	fct_orders_dd_withoutcomp	sales_rep_surr_id	bigint	none
6	fct_orders_dd_withoutcomp	city_surr_id	bigint	none
7	fct_orders_dd_withoutcomp	subchannel_surr_id	bigint	none
8	fct_orders_dd_withoutcomp	order_status	character varying(50)	none
9	fct_orders_dd_withoutcomp	returned	character varying(10)	none
10	fct_orders_dd_withoutcomp	payment_method	character varying(50)	none
11	fct_orders_dd_withoutcomp	fct_unit_price	numeric(10,2)	none
12	fct_orders_dd_withoutcomp	fct_quantity	bigint	none
13	fct_orders_dd_withoutcomp	fct_discount_percent	bigint	none
14	fct_orders_dd_withoutcomp	fct_discount_amount	numeric(10,2)	none
15	fct_orders_dd_withoutcomp	fct_shipping_cost	numeric(10,2)	none
16	fct_orders_dd_withoutcomp	fct_tax_amount	numeric(10,2)	none
17	fct_orders_dd_withoutcomp	fct_order_amount	numeric(10,2)	none
18	fct_orders_dd_withoutcomp	fct_cost_amount	numeric(10,2)	none
19	fct_orders_dd_withoutcomp	fct_profit_margin_percent	numeric(10,2)	none
20	fct_orders_dd_withoutcomp	fct_profit_amount	numeric(10,2)	none
21	fct_orders_dd_withoutcomp	fct_net_amount	numeric(10,2)	none
22	fct_orders_dd_withoutcomp	transaction_id	character varying(100)	none
23	fct_orders_dd_withoutcomp	source_system	character varying(50)	none
24	fct_orders_dd_withoutcomp	source_entity	character varying(50)	none
25	fct_orders_dd_withoutcomp	order_src_id	character varying(100)	none
26	fct_orders_dd_withoutcomp	insert_dt	date	none
27	fct_orders_dd_withoutcomp	update_dt	date	none

ANALYZE COMPRESSION command was executed to see optimal encoding proposed by Redshift

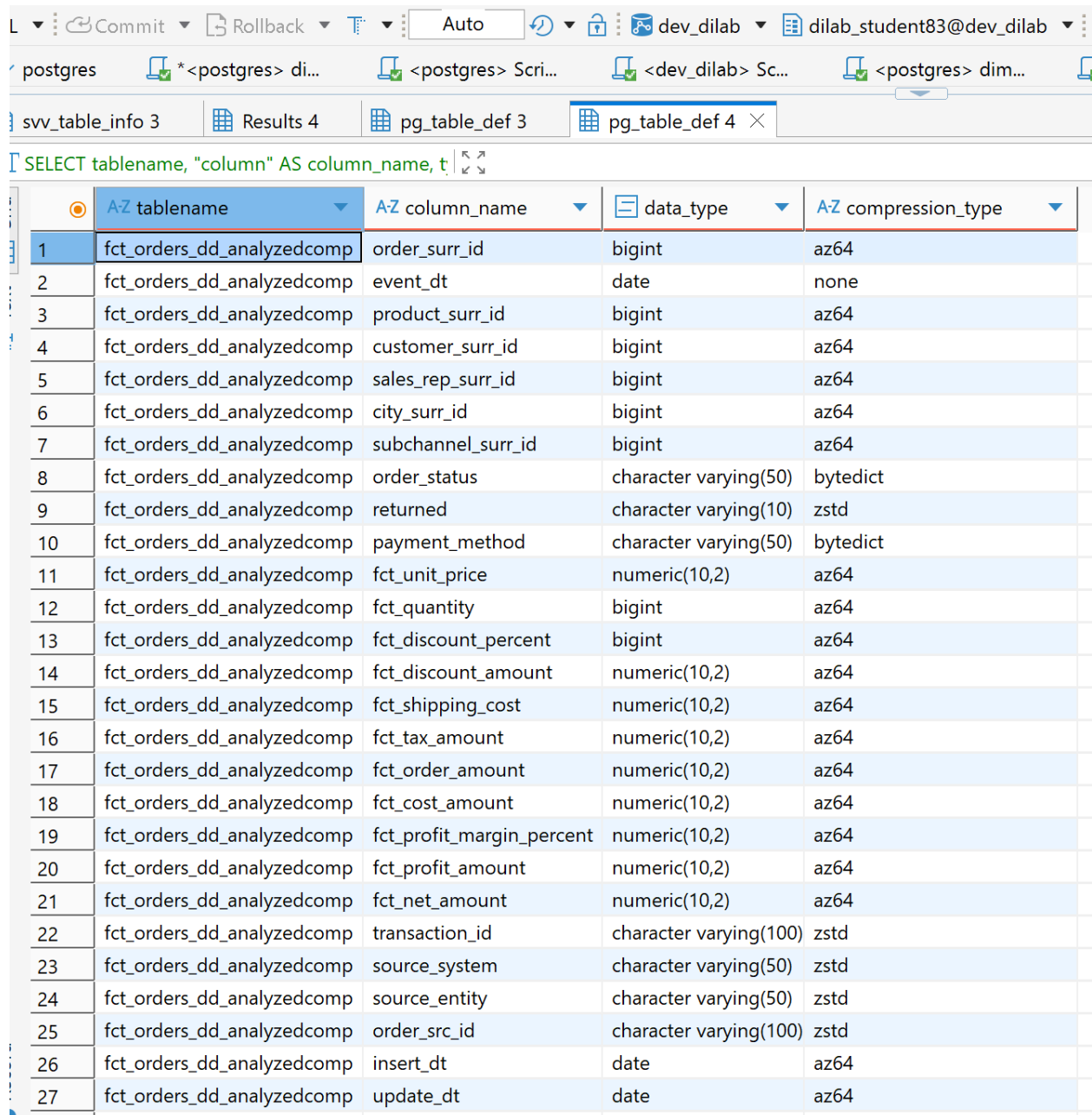
ANALYZE COMPRESSION dilab_student83.fct_ord

	A-Z Table ▼	A-Z Column ▼	A-Z Encoding ▼	A-Z Est_reduction_pct ▼
1	fct_orders_dd	order_surr_id	az64	65.67
2	fct_orders_dd	event_dt	raw	0.00
3	fct_orders_dd	product_surr_id	az64	76.59
4	fct_orders_dd	customer_surr_id	az64	99.99
5	fct_orders_dd	sales_rep_surr_id	az64	84.40
6	fct_orders_dd	city_surr_id	az64	84.40
7	fct_orders_dd	subchannel_surr_id	az64	89.08
8	fct_orders_dd	order_status	bytedict	92.04
9	fct_orders_dd	returned	zstd	94.73
10	fct_orders_dd	payment_method	bytedict	92.51
11	fct_orders_dd	fct_unit_price	az64	68.85
12	fct_orders_dd	fct_quantity	az64	91.07
13	fct_orders_dd	fct_discount_percent	az64	87.58
14	fct_orders_dd	fct_discount_amount	az64	69.75
15	fct_orders_dd	fct_shipping_cost	az64	79.88
16	fct_orders_dd	fct_tax_amount	az64	70.30
17	fct_orders_dd	fct_order_amount	az64	65.36
18	fct_orders_dd	fct_cost_amount	az64	66.02
19	fct_orders_dd	fct_profit_margin_perc	az64	77.79
20	fct_orders_dd	fct_profit_amount	az64	68.37
21	fct_orders_dd	fct_net_amount	az64	65.75
22	fct_orders_dd	transaction_id	zstd	71.71
23	fct_orders_dd	source_system	zstd	99.94
24	fct_orders_dd	source_entity	zstd	99.94
25	fct_orders_dd	order_src_id	zstd	67.45
26	fct_orders_dd	insert_dt	az64	99.99
27	fct_orders_dd	update_dt	az64	99.99

Results of analysis:

- 1) Low-Cardinality Text Columns (order_status, payment_method): Shifted from lzo (default) to bytedict (analyzed). These columns contain a small, repetitive set of distinct string values, byte-dictionary encoding replaced repeated text strings with compact integer tokens
- 2) High-Cardinality and Identifier Text Columns (transaction_id, source_system, order_src_id): Shifted from lzo (default) to zstd (analyzed). zstd provides a higher compression ratio for complex text.

Based on proposed encodings 3rd table was created



The screenshot shows a PostgreSQL database interface with a table definition for 'fct_orders_dd_analyzedcomp'. The table has 27 columns. The first 21 columns are numeric or date types, and the last 6 columns are character varying types. The compression type for each column is specified in the 'AZ compression_type' column.

	AZ tablename	AZ column_name	data_type	AZ compression_type
1	fct_orders_dd_analyzedcomp	order_surr_id	bigint	az64
2	fct_orders_dd_analyzedcomp	event_dt	date	none
3	fct_orders_dd_analyzedcomp	product_surr_id	bigint	az64
4	fct_orders_dd_analyzedcomp	customer_surr_id	bigint	az64
5	fct_orders_dd_analyzedcomp	sales_rep_surr_id	bigint	az64
6	fct_orders_dd_analyzedcomp	city_surr_id	bigint	az64
7	fct_orders_dd_analyzedcomp	subchannel_surr_id	bigint	az64
8	fct_orders_dd_analyzedcomp	order_status	character varying(50)	bytedict
9	fct_orders_dd_analyzedcomp	returned	character varying(10)	zstd
10	fct_orders_dd_analyzedcomp	payment_method	character varying(50)	bytedict
11	fct_orders_dd_analyzedcomp	fct_unit_price	numeric(10,2)	az64
12	fct_orders_dd_analyzedcomp	fct_quantity	bigint	az64
13	fct_orders_dd_analyzedcomp	fct_discount_percent	bigint	az64
14	fct_orders_dd_analyzedcomp	fct_discount_amount	numeric(10,2)	az64
15	fct_orders_dd_analyzedcomp	fct_shipping_cost	numeric(10,2)	az64
16	fct_orders_dd_analyzedcomp	fct_tax_amount	numeric(10,2)	az64
17	fct_orders_dd_analyzedcomp	fct_order_amount	numeric(10,2)	az64
18	fct_orders_dd_analyzedcomp	fct_cost_amount	numeric(10,2)	az64
19	fct_orders_dd_analyzedcomp	fct_profit_margin_percent	numeric(10,2)	az64
20	fct_orders_dd_analyzedcomp	fct_profit_amount	numeric(10,2)	az64
21	fct_orders_dd_analyzedcomp	fct_net_amount	numeric(10,2)	az64
22	fct_orders_dd_analyzedcomp	transaction_id	character varying(100)	zstd
23	fct_orders_dd_analyzedcomp	source_system	character varying(50)	zstd
24	fct_orders_dd_analyzedcomp	source_entity	character varying(50)	zstd
25	fct_orders_dd_analyzedcomp	order_src_id	character varying(100)	zstd
26	fct_orders_dd_analyzedcomp	insert_dt	date	az64
27	fct_orders_dd_analyzedcomp	update_dt	date	az64

As mentioned before, no storage reduction is observed

ble_info 3	Results 4	pg_table_def 3	svv_table_info 4	stv_blocklist(+) 5	svv_table_info 6	pg_table_def 7	fct
T "table" AS table_name, tbl_rows AS total_							
A-Z table_name	123 total_rows	123 size_in_mb	A-Z has_compression_encoding	A-Z diststyle	A-Z sortkey1		
fct_orders_dd	108,815	240	Y, AUTO(ENCODE)	KEY(order_surr_id)	event_dt		
fct_orders_dd_analyzedcomp	108,815	240	Y	KEY(order_surr_id)	event_dt		
fct_orders_dd_withoutcomp	108,815	240	N	KEY(order_surr_id)	event_dt		

As well as difference on column level

Search SQL Editor Database Window Help

SQL Commit Rollback Auto dev_dilab dilab_student83@dev_dilab

postgres * <postgres> di... <postgres> Scri... <dev_dilab> Sc... <postgres> dim... <post

'fct_orders_dd_analyzedcomp'

svv_table_info 3 Results 4 pg_table_def 3 pg_attribute 4 svv_table_info 6 pg_table_def 7

WITH blocks AS (SELECT pgc.relname AS tablena Enter a SQL expression to filter results (use Ctrl+Space)

Grid	A-Z attrname	123 sizemb_default	123 sizemb_raw	123 sizemb_analyzed	
Text	1 city_surr_id	8	8	8	
	2 customer_surr_id	8	8	8	
	3 event_dt	8	8	8	
	4 fct_cost_amount	8	8	8	
	5 fct_discount_amount	8	8	8	
	6 fct_discount_percent	8	8	8	
	7 fct_net_amount	8	8	8	
	8 fct_order_amount	8	8	8	
	9 fct_profit_amount	8	8	8	
	10 fct_profit_margin_percent	8	8	8	
	11 fct_quantity	8	8	8	
	12 fct_shipping_cost	8	8	8	
	13 fct_tax_amount	8	8	8	
	14 fct_unit_price	8	8	8	
	15 insert_dt	8	8	8	
	16 order_src_id	8	8	8	
	17 order_status	8	8	8	
	18 order_surr_id	8	8	8	
	19 payment_method	8	8	8	
	20 product_surr_id	8	8	8	
	21 returned	8	8	8	
	22 sales_rep_surr_id	8	8	8	
	23 source_entity	8	8	8	
	24 source_system	8	8	8	
	Record	25 subchannel_surr_id	8	8	8
		26 transaction_id	8	8	8
		27 update_dt	8	8	8

4. Procedure creation, distribution style and sort keys & optimization

The first execution after a change often costs more due to metadata/plan caching.

Existing distribution style of tables, its sort keys:

svv_table_info 3	Results 4	Results 3	dim_channels(+) 4	dim_channels(+) 5	pg_namespace(+) 6	pg_namespace(+) 7
SELECT n.namespace AS schema_name, c.relname AS table_name, c.diststyle AS dist_style, c.attname AS column_name, c.is_distkey AS is_distkey, c.sortkey_position AS sortkey_position						
AZ schema_name	AZ table_name	AZ dist_style	AZ column_name	is_distkey	sortkey_position	
1	dilab_student83	fct_orders_dd	order_surr_id	[v]	0	
2	dilab_student83	fct_orders_dd	event_dt	[]	1	

Only 'fct_orders_dd' has distribution and sort keys initially, 'dim_products', 'dim_channels' appears to be too small to assign it automatically.

Execution plan & duration of the initial query run

SQL	Commit	Rollback	Auto	dev_dilab	dilab_student83@dev...
pgpostgres <postgres> di... <postgres> Scri... <dev_dilab> Sc... <postgres> dim...					
svv_table_info 3	Results 4	pg_table_def 3	Results 4	svv_table_info 6	pg_table_de
EXPLAIN SELECT c.channel, p.product_category_n					
AZ QUERY PLAN					
1	XN HashAggregate (cost=6633.97..6634.15 rows=24 width=64)				
2	-> XN Hash Join DS_DIST_ALL_NONE (cost=62.58..5509.26 rows=89977 width=64)				
3	Hash Cond: ("outer".product_surr_id = "inner".product_surr_id)				
4	-> XN Hash Join DS_DIST_ALL_NONE (cost=0.07..3381.11 rows=93270 width=60)				
5	Hash Cond: ("outer".subchannel_surr_id = "inner".subchannel_surr_id)				
6	-> XN Seq Scan on fct_orders_dd f (cost=0.00..1088.15 rows=108815 width=56)				
7	-> XN Hash (cost=0.06..0.06 rows=6 width=20)				
8	-> XN Seq Scan on dim_channels c (cost=0.00..0.06 rows=6 width=20)				
9	-> XN Hash (cost=50.00..50.00 rows=5000 width=20)				
10	-> XN Seq Scan on dim_products p (cost=0.00..50.00 rows=5000 width=20)				

Execution plan was the same after altering table with compound key(same):

ALTER SORTKEY (event_dt, product_surr_id, subchannel_surr_id);

arch

SQL Editor

Database

Window

Help

SQL

Commit

Rollback

Auto

dev_dilab

dilab_student83@dev_dilab

postgres

* <postgres> di...

<postgres> Scri...

<dev_dilab> Sc...

<postgres> dim...

<postgres> fct...

```

ELSE NULL
END AS dist_style,
a.attname AS column_name.

```

pg_namespace(+) 5

Statistics 6

pg_namespace(+) 7

Results 8

dim_channels(+) 9

Results 10

EXPLAIN SELECT c.channel, p.product_category_n

Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN

1

XN HashAggregate (cost=6633.97..6634.15 rows=24 width=64)

2

-> XN Hash Join DS_DIST_ALL_NONE (cost=62.58..5509.26 rows=89977 width=64)

3

Hash Cond: ("outer".product_surr_id = "inner".product_surr_id)

4

-> XN Hash Join DS_DIST_ALL_NONE (cost=0.07..3381.11 rows=93270 width=60)

5

Hash Cond: ("outer".subchannel_surr_id = "inner".subchannel_surr_id)

6

-> XN Seq Scan on fct_orders_dd f (cost=0.00..1088.15 rows=108815 width=)

7

-> XN Hash (cost=0.06..0.06 rows=6 width=20)

8

-> XN Seq Scan on dim_channels c (cost=0.00..0.06 rows=6 width=20)

9

-> XN Hash (cost=50.00..50.00 rows=5000 width=20)

10

-> XN Seq Scan on dim_products p (cost=0.00..50.00 rows=5000 width=20)

#	Initial sort key	Compound sort key
1	0.357s	0.474s
2	0.407s	0.105s
3	0.106s	0.125s
cost	6633.97..6634.15	6633.97..6634.15

Execution time varied across runs with no consistent advantage for either sort key configuration, and planner cost remained identical (6633.97..6634.15) in both cases. This sort key choice has no measurable effect on this specific query. It may be related that size of dim tables too small.

COPY QUESTION

Shared results from responsible student:

lineorder_1 (single CSV-gzip file): 4 min 27 s

lineorder_2 (many parquet files): 1 min 47 s

Both tables hold the same data (100,000,008 rows)

Explanations:

The lineorder_2 loaded faster despite identical row counts. That may be explained by:

1) the single gzip-compressed CSV file cannot be split across slices, forcing the load one slice without parallel execution, whereas the multiple Parquet files were distributed and loaded across slices;

2) Parquet's format - binary, pre-typed columnar - avoids the per-row CSV parsing and type-casting overhead that the gzip-CSV has.

WORKING WITH EXTERNAL TABLES

1. External table is created

```
SELECT * FROM svv_external_schemas
WHERE schemaname = 'user_dilab_student83_ext';
```

esoid	123 eskind	AZ schemaname	123 esowner	databasename	esoptions
921410	1	user_dilab_student83_ext	474	n_diachenko_dwh_db	{ "IAM_ROLE": "arn:aws:iam::260586643565:role/dilab-

2. Data which contain date column exported into S3 in a way, so each subfolder contains 1 month data

Amazon S3 > Buckets > nadiia-diachenko-dwh-bucket > di_dwh_database/ > fct_orders_dd_partitioned/

Amazon S3

- Buckets
 - General purpose buckets
 - Directory buckets
 - Table buckets
 - Vector buckets
- Files
 - File systems
- Access management and security
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer
- Storage management and insights
 - Storage Lens
 - Batch Operations

fct_orders_dd_partitioned/

Objects (6)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicit permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	event_dt_month=2026-02/	Folder	-	-	-
☐	event_dt_month=2026-03/	Folder	-	-	-
☐	event_dt_month=2026-04/	Folder	-	-	-
☐	event_dt_month=2026-05/	Folder	-	-	-
☐	event_dt_month=2026-06/	Folder	-	-	-
☐	event_dt_month=2026-07/	Folder	-	-	-

3. PARTITIONED external table is created

```
SELECT *
FROM svv_external_tables
WHERE schemaname = 'user_dilab_student83_ext'
AND tablename = 'ext_student83_partitioned';
```

AZ redshift_database_name	AZ schemaname	AZ tablename	AZ location	AZ input_format
dev_dilab	user_dilab_student83_ext	ext_student83_partitioned	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive

```
SELECT * FROM svv_external_partitions
WHERE schemaname = 'user_dilab_student83_ext'
```

AZ schemaname	AZ tablename	AZ values	AZ location	AZ input_format
user_dilab_student83_ext	ext_student83_partitioned	["2026-02"]	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive.q1.io.parquet.Map
user_dilab_student83_ext	ext_student83_partitioned	["2026-03"]	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive.q1.io.parquet.Map
user_dilab_student83_ext	ext_student83_partitioned	["2026-04"]	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive.q1.io.parquet.Map
user_dilab_student83_ext	ext_student83_partitioned	["2026-05"]	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive.q1.io.parquet.Map
user_dilab_student83_ext	ext_student83_partitioned	["2026-06"]	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive.q1.io.parquet.Map
user_dilab_student83_ext	ext_student83_partitioned	["2026-07"]	s3://nadiia-diachenko-dwh-bucket/di_dwh_database	org.apache.hadoop.hive.q1.io.parquet.Map

4. Data comparison

```
WITH original_counts AS (
    SELECT
        TO_CHAR(event_dt, 'YYYY-MM') AS event_dt_month,
        COUNT(*) AS original_row_count
    FROM dilab_student83.fct_orders_dd
    GROUP BY 1
),
external_counts AS (
    SELECT
        event_dt_month,
        COUNT(*) AS external_row_count
    FROM user_dilab_student83_ext.ext_student83_partitioned
    GROUP BY 1
)
SELECT
    COALESCE(o.event_dt_month, e.event_dt_month) AS month,
    o.original_row_count,
    e.external_row_count,
    COALESCE(o.original_row_count, 0) - COALESCE(e.external_row_count, 0) AS difference
FROM original_counts o
FULL OUTER JOIN external_counts e
    ON o.event_dt_month = e.event_dt_month
ORDER BY month;
```

Statistics 1 | stv_external_partitions 2 | original_counts(+) 3 X

Filter on original_counts AS (SELECT TO_CHAR(event_dt, 'YYYY-MM') AS event_dt_month, COUNT(*) AS original_row_count FROM dilab_student83.fct_orders_dd GROUP BY 1) Enter a SQL expression to filter results (use Ctrl+Space)

month	original_row_count	external_row_count	difference
2026-02	21,779	21,779	0
2026-03	24,548	24,548	0
2026-04	23,781	23,781	0
2026-05	24,425	24,425	0
2026-06	14,257	14,257	0
2026-07	25	25	0

5. Query plan

```
EXPLAIN
SELECT *
FROM user_dilab_student83_ext.ext_student83_partitioned
WHERE event_dt_month = '2026-03';
```

Statistics 1 | stv_external_partitions 2 | Results 3 X

Filter on SELECT * FROM user_dilab_student83_ext.ext_student83_partitioned WHERE event_dt_month = '2026-03'; Enter a SQL expression to filter results (use Ctrl+Space)

QUERY PLAN

- XN Partition Loop (cost=0.00..25.57 rows=545 width=959)
 - > XN Seq Scan PartitionInfo of user_dilab_student83_ext.ext_student83_partitioned (cost=0.00..12.50 rows=5 width=25)
 - Filter: ((event_dt_month)::text = '2026-03':text)
 - > XN S3 Query Scan ext_student83_partitioned (cost=0.00..2.18 rows=109 width=934)
 - > S3 Seq Scan user_dilab_student83_ext.ext_student83_partitioned location: "s3://nadiia-diachenko-dwh-bucket/di_dwh_database/fct_orders_dd_partitioned" format:PARQUET (cost=0.00..1.09 rows=109 width=934)

Querying ext_student83_partitioned with WHERE event_dt_month = '2026-03' produces a plan with two stages: a PartitionInfo scan that evaluates the filter against partition metadata first (isolating the matching partition before any data access), followed by an S3 Query Scan that

reads only the Parquet files under the matched event_dt_month=2026-03/ prefix. This confirms partition pruning is functioning — the query reads data proportional to one month's data (109 rows in this cost estimate) rather than scanning all 6 months' files, which is the core cost and performance benefit of external table partitioning.